

РОЗБІР НАВЧАЛЬНОГО ПРОЄКТУ

my-blog: ЯК ЦЕ ВЛАШТОВАНО

Покроковий гайд для фронтендера, який хоче зрозуміти бекенд — від першого рядка Express до HTTPS у Docker Compose. Не «як написати блог», а **чому кожен інструмент тут саме такий**.

Стек: Node.js · Express · PostgreSQL · Prisma ORM · Next.js · Zustand · Docker · Docker Compose · NGINX · Jest · Supertest

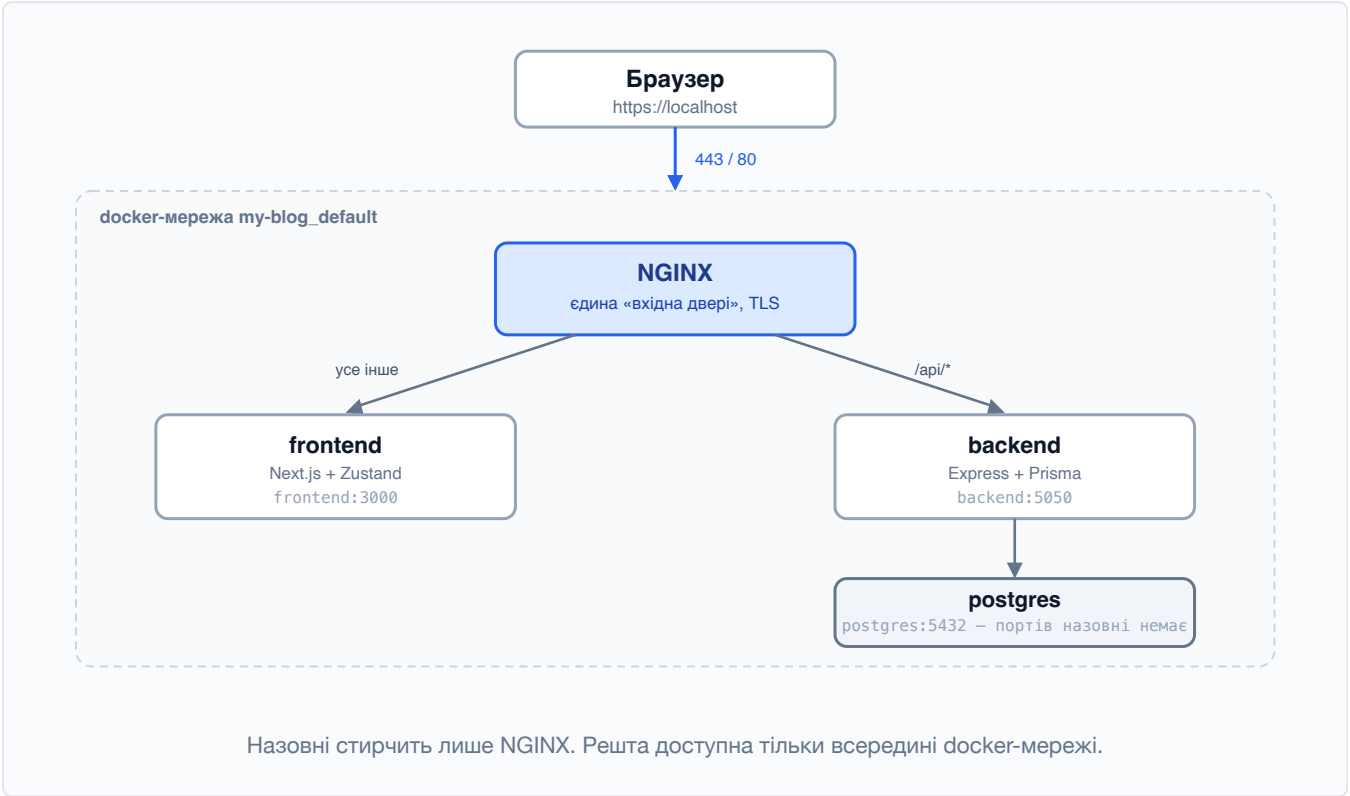
Зміст

- 01 **Карта проєкту** — хто з ким говорить і чому саме так
- 02 **Node.js і Express** — що таке сервер і з чого він складається
- 03 **PostgreSQL** — чому реляційна база, а не файл із JSON
- 04 **Prisma ORM** — схема, міграції, клієнт
- 05 **Автентифікація** — bcrypt, JWT і middleware
- 06 **CRUD і права доступу** — хто що може редагувати
- 07 **Фронтенд** — Next.js, Zustand і чому ми прибрати axios
- 08 **Docker** — образ, контейнер, том — на пальцях
- 09 **Docker Compose** — чотири сервіси однією командою
- 10 **NGINX і HTTPS** — reverse proxy, статика, 443 порт
- 11 **Тести** — юніт проти інтеграційних, на конкретиці
- 12 **Життя одного запиту** — від кліку до рядка в базі
- 13 **Габлі** — чотири реальні баги цього проєкту
- 14 **Шпаргалка** — команди, які знадобляться щодня

01 Карта проєкту

Перш ніж читати код, треба тримати в голові картинку: що це за коробки і чому їх чотири, а не одна.

Застосунок складається з **чотирьох процесів**, кожен у своєму контейнері. Вони не знають одне про одного нічого, крім імені та порту.



Хто за що відповідає

Сервіс	Відповідає за	Якби його не було
nginx	Приймає HTTPS, вирішує, кому віддати запит, віддає статику зі стисненням	Довелось би вшити TLS і роздачу файлів на Node — він для цього повільніший і це зайва відповідальність
frontend	Малює HTML і React-компоненти, тримає стан користувача	Немає інтерфейсу
backend	Бізнес-логіка: хто має право створити пост, чи правильний пароль	Логіка поїхала б у браузер, де її може підмінити будь-хто
postgres	Зберігає дані так, щоб вони пережили рестарт	Дані зникали б разом із процесом

Чому не один процес

Технічно Next.js уміє і бекенд (API routes), і тоді контейнер був би один. Але поділ дає три речі: кожен частину можна масштабувати окремо, впала одна — інші живі, і бекенд можна переписати хоч на Go, не чіпаючи фронт. Для навчального проєкту головне інше: **ти бачиш межі відповідальності руками**, а не в теорії.

Структура тек

```
my-blog/
├── my-express-app/    ← бекенд: Express + Prisma
│   ├── src/
│   ├── prisma/       ← схема БД і міграції
│   ├── tests/        ← юніт + інтеграційні
│   └── Dockerfile
├── my-blog-frontend/ ← Next.js
│   ├── src/
│   └── Dockerfile
├── nginx/
│   ├── nginx.conf    ← конфіг проксі й TLS
│   └── Dockerfile
└── docker-compose.yml ← склеює все докупи
```

02 Node.js і Express

Node — це той самий JavaScript, тільки без браузера. Express — тонка бібліотека, яка перетворює «сирий» HTTP на зрозумілі роути.

Що таке сервер насправді

Сервер — це процес, який **не завершується**. Він відкриває порт і сидить у циклі: прийшов запит → обробив → відповів → чекає далі. Без Express це виглядало б так:

```
const http = require('http')
http.createServer((req, res) => {
  // і тут ти сам парсиш URL, метод, тіло, заголовки...
  if (req.url === '/api/posts' && req.method === 'GET') { ... }
}).listen(5050)
```

Express прибирає цю рутину. Він дає три поняття, і на них тримається весь бекенд:

Поняття	Що це	Де в проєкті
Роут	Правило «метод + шлях → функція»	src/routes/
Контролер	Функція, яка щось робить і відповідає	src/controllers/
Middleware	Функція «між» — бачить запит до контролера і може його не пустити	src/middlewares/

Складання застосунку

Ключова деталь, яку легко проґавити: у нас **два файли замість одного**.

src/app.ts

```
const app = express()

app.use(cors())           // дозволяє запити з іншого origin
app.use(express.json())   // перетворює тіло запиту на об'єкт

app.use('/api/auth', authRoutes)
app.use('/api/posts', postsRoutes)
app.use('/api/comments', commentsRoutes)

export default app // ← застосунок є, але порт не відкритий
```

src/server.ts

```
import app from './app'
app.listen(PORT) // ← ось тепер відкриваємо порт
```

Навіщо цей поділ

Щоб тести могли взяти `app` і слати в нього запити **без реального порту**. Якби `listen()` був у тому ж файлі, кожен тестовий файл намагався б зайняти 5050 і вони конфліктували б між собою. Це стандартний прийом, і робити його краще з першого дня.

Порядок `middleware` має значення

Express виконує їх зверху вниз. `express.json()` стоїть до роутів — інакше в контролері `req.body` був би `undefined`. Так само `authMiddleware` стоїть перед контролером:

```
src/routes/posts.routes.ts
```

```
router.get('/', getAllPosts) // публічно
router.post('/', authMiddleware, createPost) // спершу перевірка токена
```

Якщо `authMiddleware` викличе `res.status(401)` і **не** викличе `next()` — ланцюжок обірветься, і `createPost` просто не запуститься.

03 PostgreSQL

База — це не «місце, куди кладуть дані». Це система, яка гарантує, що дані лишаться коректними, навіть коли все пішло не так.

Що дає реляційна база, чого не дасть JSON-файл

- **Зв'язки і цілісність.** У коментаря є `postId`. База фізично не дозволить створити коментар до неіснуючого поста — це *foreign key*. У файлі довелося би перевіряти руками й усе одно проґавити.
- **Каскади.** Видалив пост — його коментарі зникли самі. Один рядок у схемі замість циклу в коді.
- **Унікальність.** `email @unique` означає, що двох однакових email не буде *ніколи*, навіть якщо два запити прийшли в одну мілісекунду.
- **Транзакції.** Або всі зміни застосувались, або жодна. Немає стану «половина записалась».
- **Паралельність.** Десятки запитів одночасно — це нормальний режим роботи, а не проблема.

Термін

ACID — чотири гарантії, за які й обирають такі бази: атомарність, узгодженість, ізолюваність, довговічність. Простими словами: «якщо база сказала *записано* — значить записано, і воно не суперечить решті даних».

Три таблиці цього блогу

Таблиця	Поля	Зв'язки
User	id, email (unique), password (хеш), name, createdAt	має багато Post і Comment
Post	id, title, content, published, authorId, createdAt, updatedAt	належить User, має багато Comment
Comment	id, content, postId, authorId, createdAt, updatedAt	належить і Post, і User

04 Prisma ORM

ORM — перекладач між об'єктами в коді й таблицями в базі. Prisma додає до цього ще дві важливі речі: типи і міграції.

Схема — єдине джерело правди

prisma/schema.prisma

```
model Post {
  id          Int          @id @default(autoincrement())
  title       String
  content     String
  author      User         @relation(fields: [authorId], references: [id], onDelete: Cascade)
  authorId    Int
  comments    Comment[]
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}
```

З цього одного файлу Prisma робить три різні речі:

1 SQL-міграцію

`prisma migrate dev` порівнює схему з попереднім станом і генерує файл `.sql` у `prisma/migrations/`. Це звичайний текстовий файл, який лягає в `git` — тобто **історія структури бази версіонується разом із кодом**.

2 Типізований клієнт

`prisma generate` створює код у `node_modules/@prisma/client`. Після цього редактор знає, що в поста є `title`, і підкреслить `titel` червоним ще до запуску.

3 Валідацію на етапі компіляції

Спроба записати рядок у числове поле — помилка TypeScript, а не 500-та на проді.

Дві команди, які легко переплутати

команда	коли	що робить
<code>migrate dev</code>	у розробці, коли мінєш схему	створює новий файл міграції і застосовує його
<code>migrate deploy</code>	на проді / в контейнері	тільки застосовує вже наявні міграції, нічого не вигадує

Реальні граблі цього проекту

У Docker міграції не запускались **ніколи**. Контейнер із базою піднімався порожній, Prisma-клієнт стукав у таблицю `User`, якої не існує, і кожен запит повертав `500`. Локально все працювало, бо там міграції колись накотили руками. Лікування — один рядок у `docker-compose.yml`:

```
command: sh -c "npx prisma migrate deploy && npm start"
```

`migrate deploy` ідемпотентний: якщо міграції вже накотились, він просто нічого не робить. Тому його безпечно ганяти при кожному старті.

Один клієнт на застосунок

`src/lib/prisma.ts`

```
const prisma = new PrismaClient()
export default prisma
```

Спочатку в проєкті було три `new PrismaClient()` — по одному в кожному контролері. Кожен відкриває власний пул з'єднань до бази, і на кількох контролерах це вже десятки зайвих коннектів. Тепер він один. Бонусом це **єдина точка, яку зручно підмінити в тестах** — про це в розділі 11.

05 Автентифікація

Дві окремі задачі, які часто плутають: **автентифікація** — «хто ти», **авторизація** — «що тобі можна».

Крок 1. Пароль ніколи не зберігається

```
const hashedPassword = await bcrypt.hash(password, 10)
```

У базі лежить **хеш** — результат односторонньої функції. З хешу пароль не відновити навіть маючи повний дамپ бази. Перевірка йде в інший бік: `bcrypt.compare(введений, хеш)` хешує введене й порівнює результати.

Що означає «10»

Це *cost factor* — `bcrypt` зробить 2^{10} ітерацій. Хешування навмисне повільне (~100 мс), щоб перебір паролів був дорогим. Кожна +1 подвоює час. Ще `bcrypt` сам додає *сіль* — випадковий рядок до пароля, тож однакові паролі двох користувачів дають різні хеші.

Крок 2. Токен замість сесії

```
const token = jwt.sign(
  { userId: user.id, email: user.email }, // payload
  process.env.JWT_SECRET,                // підпис
  { expiresIn: '24h' }
)
```

JWT — це три частини через крапку: заголовок, payload, підпис. Payload **не зашифрований**, його можна прочитати на `jwt.io`. Захищає не таємність, а підпис: змінити `userId` в токені можна, але без `JWT_SECRET` перерахувати підпис — ні, і сервер такий токен відкине.

Наслідок, про який забувають

Токен **не можна відкликати**. Сервер не тримає список виданих токенів — у цьому й суть «stateless». Якщо токен вкрали, він працює до закінчення терміну. Тому 24 години, а не рік. Тому ж не варто класти в payload нічого зайвого.

Крок 3. Middleware як турнікет

```
src/middlewares/auth.middleware.ts
```

```
const authHeader = req.headers.authorization
if (!authHeader?.startsWith('Bearer '))
  return res.status(401).json({ error: 'Потрібен токен' })

try {
  req.user = jwt.verify(authHeader.substring(7), process.env.JWT_SECRET)
  next() // ← пропускаємо далі
} catch {
  res.status(401).json({ error: 'Невірний токен' }) // ← обриваємо
}
```

Після нього кожен контролер може довіряти `req.user.userId`. Це принципово: **ID автора береться з токена, а не з тіла запиту**. Інакше будь-хто надіслав би `{"authorId": 1}` і писав пости від чужого імені.

401 проти 403

401 Unauthorized — «я не знаю, хто ти» (немає або битий токен). **403 Forbidden** — «я знаю, хто ти, і тобі не можна» (чужий пост). Плутанина між ними — класика на співбесідах.

06 CRUD і права доступу

CRUD — Create, Read, Update, Delete. У REST кожна операція має свій HTTP-метод, і це не формальність: від методу залежить поведінка кешів, проксі й браузера.

Метод	Роут	Дія	Токен
GET	/api/posts	список постів	не треба
GET	/api/posts/:id	пост із коментарями	не треба
POST	/api/posts	створити	треба
PUT	/api/posts/:id	оновити	треба + бути автором
DELETE	/api/posts/:id	видалити	треба + бути автором

Перевірка прав — завжди у два кроки

```
const post = await prisma.post.findUnique({ where: { id: Number(id) } })

if (!post) return res.status(404).json({ error: 'Пост не знайдено' })
if (post.authorId !== req.user.userId)
  return res.status(403).json({ error: 'Тільки автор може редагувати' })

const updated = await prisma.post.update({ ... })
```

Спершу «чи існує», потім «чи твоє», і тільки тоді дія. Пропустити першу перевірку — отримати незрозумілу помилку Prisma замість чесної 404.

Цікавий нюанс із коментарями

Видалити коментар може **або його автор, або автор поста** — це модерація у себе під постом. А от *редагувати* чужий коментар не може ніхто, навіть автор поста: інакше можна було б підмінити чиїсь слова. Різні правила для різних дій — нормально, і саме такі місця варто покривати тестами.

07 Фронтенд: Next.js, Zustand і fetch

Фронтенд тут навмисне тонкий: показати дані й зібрати форму. Уся логіка прав — на бекенді.

Zustand — стан у трьох рядках

Потреба проста: після логіну **кілька сторінок** мають знати, хто зайшов. Прокидати це пропсами — боляче, Redux — надлишково.

```
export const useAuthStore = create<AuthState>((set) => ({
  user: null,
  token: null,
  login: (user, token) => {
    set({ user, token }) // у пам'ять
    localStorage.setItem('blog-token', token) // щоб пережити F5
  },
}))
```

Стан у пам'яті зникає при перезавантаженні сторінки — тому дубль у `localStorage` і відновлення сесії в `useEffect` при старті.

Чому ми викинули axios

Axios дає: базовий URL, JSON, інтерсептори, помилку на 4xx. Порахуймо, скільки з цього **не вміє** нативний `fetch`, який уже є в кожному браузері:

Задача	fetch сам	рішення
Базовий URL	ні	шаблонний рядок
JSON у тілі	ні	<code>JSON.stringify</code> + заголовок
Заголовок з токеном	ні	читаємо <code>localStorage</code> у функції
Помилка на 4xx/5xx	ні	перевірити <code>res.ok</code> і кинути самим

Разом — близько 60 рядків у `src/lib/api.ts`. Натомість мінус одна залежність із власним деревом транзитивних пакетів.

Аргумент про supply chain

Кожен прм-пакет — це чужий код, який виконується у твоєму білді з тими ж правами, що й твій власний. Атаки останніх років працюють не через дірку в коді пакета, а через **захоплений акаунт мейнтейнера**: виходить нова патч-версія зі шкідливим `postinstall`, і всі, у кого `^` у версії, підтягують її автоматично. Захист простий і нудний: менше залежностей, лок-файл у git, оновлення усвідомлено. Найнадійніша залежність — та, якої немає.

Ключова деталь fetch-обгортки

```
const res = await fetch(`${BASE_URL}${path}`, { method, headers, body })

// 204 після DELETE не має тіла — res.json() тут би впав
const payload = res.status === 204 ? null : await res.json()

// fetch, на відміну від axios, на 4xx/5xx помилку НЕ кидає
if (!res.ok) throw new ApiError(res.status, payload)
```

Найчастіша помилка новачка з fetch

`await fetch(...)` **не кидає виняток** на 404 чи 500 — проміс успішно резолвиться. `catch` спрацює лише коли мережа не відповіла взагалі. Тому перевірка `res.ok` обов'язкова, інакше застосунок мовчки працюватиме з тілом помилки замість даних.

08 Docker

Docker вирішує рівно одну проблему — «у мене працює». Але спершу три терміни, які постійно плутають.

Термін	Аналогія	На практиці
Образ	клас	незмінний зліпок: ОС + Node + твій код
Контейнер	об'єкт	запущений процес із того образу; видалив — усі зміни всередині зникли
Том (volume)	зовнішній диск	тека, яка переживає видалення контейнера — там дані бази

Контейнер — це не віртуалка

Віртуалка запускає повноцінну гостьову ОС (гігабайти, десятки секунд). Контейнер — це звичайний процес на ядрі хоста, якому обмежили видимість файлової системи, мережі та процесів. Звідси і швидкість старту, і легкість образів.

Dockerfile крок за кроком

my-express-app/Dockerfile

```
FROM node:18-slim          // 1. з чого починаємо
WORKDIR /app                // 2. робоча тека всередині
COPY package*.json ./       // 3. спершу ТІЛЬКИ маніфести
COPY prisma ./prisma/
RUN npm install              // 4. ставимо залежності
RUN npx prisma generate      // 5. генеруємо Prisma-клієнт
COPY . .                    // 6. і аж тепер увесь код
RUN npm run build             // 7. TypeScript → JavaScript
CMD ["npm", "start"]         // 8. що запускати
```

Чому package.json копіюється окремо

Кожен рядок Dockerfile — шар із кешем. Шар перебудовується, тільки якщо змінились його вхідні дані. Якби `COPY . .` стояв перед `npm install`, то **будь-яка** правка в коді інвалідувала б кеш і тягнула повний `npm install` — це хвилини на кожен білд. А так залежності перевстановлюються лише коли реально змінився `package.json`.

.dockerignore — не косметика

Його не було, і це справжня міна. `COPY . .` копіює **все**, включно з локальним `node_modules`, і затирає ті, що щойно поставились у контейнері. А там нативні бінарники: Prisma engine і bcrypt, зібрані під macOS, всередині Linux-контейнера просто не запускаються.

```
node_modules // бінарники з хоста – під іншу ОС
dist
.env // секрети в образі не потрібні
tests // у продакшн-образ не їдуть
```


09 Docker Compose

Один Dockerfile = один контейнер. Compose описує **систему** з чотирьох контейнерів у єдиному YAML.

Compose бере на себе три речі, які інакше довелось би робити руками:

- **Мережа.** Створює спільну мережу, де кожен сервіс доступний за *іменем*: бекенд ходить у базу за адресою `postgres:5432`, без жодних IP.
- **Порядок і готовність.** `depends_on` + `healthcheck` — бекенд стартує не «після запуску» бази, а після того, як вона реально **готова приймати з'єднання**.
- **Час життя даних.** Іменовані томи переживають `down` і перезбирання образів.

Сервіс бази — і головне рішення в ньому

```
postgres:
  image: postgres:15
  environment:
    POSTGRES_PASSWORD: password
    POSTGRES_DB: blog
  expose:
    - "5432"           // ← видно тільки всередині docker-мережі
  volumes:
    - pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ['CMD-SHELL', 'pg_isready -U postgres -d blog']
```

ports проти expose — різниця критична

Було `ports: - "5432:5432"`. Це означає: **прокинути порт контейнера на хост**. А оскільки Docker сам прописує правила у firewall, на сервері з білою IP така база стає доступною з інтернету — з паролем `password` із `docker-compose.yml`. Це один із найпоширеніших способів злити базу.

`expose` нічого не прокидає, це лише декларація порту всередині docker-мережі. Бекенд ходить у базу як і раніше, а ззовні її **не існує**. Перевірити: `docker compose port postgres 5432` має не повернути нічого.

Правило просте: назовні стирчить **тільки** те, до чого має ходити браузер. Тут це NGINX і більше нічого.

Фронтенд: чому саме build-arg, а не environment

```
frontend:
  build:
    context: ./my-blog-frontend
    args:
      NEXT_PUBLIC_API_URL: /api
```

Next.js **вшиває** змінні з префіксом `NEXT_PUBLIC_` прямо у клієнтський бандл під час білду — це буквально текстова заміна. Передати їх через `environment` у compose марно: бандл уже зібрано, і в браузері вони будуть `undefined`.

Саме це тут і сталося: фронт мовчки падав на запасний варіант `http://localhost:5050` — а з `https`-сторінки браузер такий запит заблокував би як `mixed content`. Значення `/api` (відносний шлях) вирішує це остаточно: запит іде на той самий `origin`, тобто в NGINX, і працює однаково на `http` і `https`.

10 NGINX і HTTPS

NGINX стоїть перед усім і вирішує, кому віддати запит. Це **reverse proxy**: клієнт бачить один сервер, а за ним ховається скільки завгодно.

Що він робить у цьому проєкті

1. **Термінує TLS** — шифрування закінчується на ньому, далі по внутрішній мережі йде звичайний http.
2. **Маршрутизує** — `/api/*` у бекенд, решта у фронтенд.
3. **Прибирає CORS як проблему** — для браузера це один origin, отже cross-origin запитів немає взагалі.
4. **Віддає статику** зі стисненням і кешем.

```
location /api/ { proxy_pass http://backend; } // у Express
location /     { proxy_pass http://frontend; } // у Next.js
```

Статика: чому це помітно

```
location /_next/static/ {
    proxy_pass http://frontend;
    add_header Cache-Control "public, max-age=31536000, immutable";
}
gzip on;
```

Імена файлів у `/_next/static/` містять хеш вмісту: змінився код — змінилось ім'я. Тому їх можна кешувати на рік без ризику показати стару версію. `immutable` каже браузеру навіть не перепитувати. Плюс `gzip` — текст стискається в рази.

HTTPS: 443 порт

Було тільки `listen 80` — тобто весь трафік ішов відкритим текстом, включно з паролями у формі логіну та JWT-токенами в заголовках. Стало:

```
server {
    listen 80;
    location / { return 301 https://$host$request_uri; } // жорсткий редірект
}

server {
    listen 443 ssl;
    http2 on;

    ssl_certificate      /etc/nginx/certs/server.crt;
    ssl_certificate_key  /etc/nginx/certs/server.key;
    ssl_protocols        TLSv1.2 TLSv1.3;

    add_header Strict-Transport-Security "max-age=31536000" always;
}
```

Директива	Навіщо
return 301 https://	випадковий http-запит не піде відкритим текстом
http2 on	кілька файлів в одному з'єднанні; доступний лише поверх TLS
ssl_protocols 1.2 1.3	старі версії протоколу мають відомі дірки — вимикаємо
Strict-Transport-Security	браузер рік не ходитиме сюди по http навіть за прямим посиланням

Звідки береться сертифікат

Сертифікат — це файл, яким сервер доводить, що він саме той, за кого себе видає. Справжній видає центр сертифікації (Let's Encrypt — безкоштовно), але він вимагає **реального домену**. Для `localhost` такий не отримати, тому тут *self-signed* — сервер підписує сам себе.

Щоб `docker compose up` працював одразу після клону, сертифікат генерується сам при першому старті контейнера:

```
// nginx/10-generate-self-signed-cert.sh
if [ -f "$CERT_FILE" ]; then exit 0; fi // є справжній — не чіпаємо
openssl req -x509 -nodes -newkey rsa:2048 -days 365 ...
```

Офіційний образ `nginx` сам виконує все з теки `/docker-entrypoint.d/` перед стартом — зручний гачок. Приватні ключі при цьому **не потрапляють у git**, вони живуть у `docker-томі`.

Чому браузер лається

На *self-signed* сертифікат буде попередження «Not secure» — це нормально й очікувано: браузер не знає того, хто його підписав. Шифрування при цьому **справжнє**. Для продакшну міняється рівно одне: замість самопідписаного кладеш сертифікат від Let's Encrypt у ту саму теку. Конфіг чіпати не треба — під це в ньому вже лишений `location /.well-known/acme-challenge/`.

11 Тести

Два види тестів відповідають на різні питання. Плутати їх — типова помилка, від якої тести стають повільними й ненадійними.

	Юніт	Інтеграційні
Питання	чи правильна логіка функції	чи працюють частини разом
База	замокана, її немає	справжній PostgreSQL
Швидкість	~3 с на 32 тести	~9 с на 27 тестів
Ловлять	помилки в гілках if, забуті перевірки прав	биті роути, зламані зв'язки в БД, каскади
Не ловлять	що роут не підключений у app.ts	рідкісні гілки — їх довго готувати

Юніт: підміна Prisma

Ось тут і окупається єдиний `src/lib/prisma.ts`: досить підмінити **один** модуль, і жоден контролер не піде в базу.

```
jest.mock('../src/lib/prisma', () => ({
  default: require('../helpers/prismaMock').prismaMock,
}))

it('повертає 403, якщо редагує не автор', async () => {
  prismaMock.post.findUnique.mockResolvedValue({ id: 3, authorId: 1 })

  await updatePost(mockRequest({ user: { userId: 2 } }), res)

  expect(prismaMock.post.update).not.toHaveBeenCalled() // ← головне
  expect(res.status).toHaveBeenCalledWith(403)
})
```

Зверни увагу на передостанній рядок: перевіряємо не лише код відповіді, а й що запису в базу **не сталося**. Тест, який дивиться тільки на 403, пропустив би баг «відповіли 403, але дані все одно змінили».

Інтеграційні: справжній HTTP і справжня база

```
const res = await request(app) // supertest — той самий app
  .post('/api/posts')
  .set('Authorization', author.auth)
  .send({ title: 'Пост', content: 'Текст' })
  .expect(201)

const inDb = await prisma.post.findUnique({ where: { id: res.body.id } })
expect(inDb.title).toBe('Пост') // перевіряємо саму базу
```

Тут проходить увесь ланцюжок: роутинг → `express.json()` → middleware з перевіркою токена → контролер → Prisma → PostgreSQL. Токен при цьому **справжній** — отриманий через реальний `/api/auth/login`.

Три правила, щоб тести не «пливли»

1 Окрема база, ніколи не робоча

`docker-compose.test.yml` піднімає `blog_test` на порту 5433, дані в `tmpfs` — тобто в пам'яті й зникають безслідно. Плюс запобіжник у `setup`: якщо в `DATABASE_URL` немає `blog_test` — тести падають, не запустившись.

2 Чиста база перед кожним тестом

```
beforeEach(async () => {  
  await prisma.comment.deleteMany() // порядок важливий:  
  await prisma.post.deleteMany()    // спершу залежні,  
  await prisma.user.deleteMany()    // потім батьківські  
})
```

Інакше тести починають залежати від порядку запуску — і падають лише іноді. Це найгірший вид падінь.

3 Послідовно, не паралельно

`maxWorkers: 1` — усі тести ділять одну базу, паралельні воркери затирали б дані одне одному.

Що запускати

```
npm test           // юніт, без бази — хоч щохвилини  
npm run test:integration // сам підніме базу і накопить міграції  
npm run test:all    // перед комітом
```

12 Життя одного запиту

Найкорисніша вправа для розуміння системи: провести один запит крізь усі шари. Беремо «користувач створює пост».

1 Браузер · React

`onSubmit` у формі викликає `api.post('/posts', { title, content })`.

2 Браузер · обгортка над `fetch`

Допишує базовий URL (`/api`), додає `Content-Type: application/json`, дістає токен із `localStorage` і ставить `Authorization: Bearer ...`. Тіло йде через `JSON.stringify`.

3 Мережа · TLS

`POST https://localhost/api/posts`. Запит зашифровано — у Wi-Fi видно лише те, що ти кудись ходив, але не що саме надіслав.

4 NGINX

Розшифровує, бачить префікс `/api/`, підставляє `X-Forwarded-For` та `X-Forwarded-Proto` і передає далі на `backend:5050` — уже по внутрішній мережі, без шифрування.

5 Express · middleware

`express.json()` перетворює тіло на об'єкт. `authMiddleware` перевіряє підпис токена і кладе `req.user = { userId: 42 }`. Немає токена — `401`, і на цьому все закінчується.

6 Контролер

`createPost` бере `title` і `content` із тіла, а `authorId` — з токена, не з тіла. Це і є та межа, де закінчується довіра до клієнта.

7 Prisma → PostgreSQL

`prisma.post.create()` перетворюється на `INSERT INTO "Post" ...`. База перевіряє foreign key на `authorId` і повертає створений рядок із `id`.

8 Назад

`201 Created` + JSON поста → NGINX шифрує → обгортка бачить `res.ok` і віддає розпарсене тіло → React викликає `fetchPosts()`, і пост з'являється у списку.

Де саме «живе» безпека

На кроці 5 (чи справжній токен) і кроці 6 (`authorId` береться з токена). Усе, що зробив браузер до цього, — лише *прохання*. Перевірки на фронтенді потрібні для зручності, але захищає тільки бекенд: DevTools відкриті у всіх.

13 Граблі цього проєкту

Чотири реальні баги, знайдені при доведенні проєкту до ладу. Усі — типові, і саме тому корисні.

1. Міграції не запускались у Docker

Симптом: локально все працює, у контейнерах будь-який запит — `500`.

Причина: `docker compose up` піднімав порожню базу, а `prisma migrate` не викликав ніхто.

Урок: локальна база вже мала таблиці з давніх ручних запусків — і це маскувало проблему.

Перевіряти треба на **чистому томі**: `docker compose down -v`.

2. Реєстрація зберігала `undefined` замість токена

Симптом: після реєстрації користувач наче залогінений, але перше ж створення поста — `401`.

Причина: `/auth/register` повертає лише дані користувача, без токена. Фронт же робив `login(res.user, res.token)`, і в `localStorage` лягав `undefined`.

Урок: у JS звернення до неіснуючого поля не падає, а тихо дає `undefined`. Такі баги ловляться тестом, який перевіряє не «чи не було помилки», а **що саме опинилось у сховищі**.

3. `node_modules` з macOS усередині Linux-контейнера

Симптом: образ збирається, але падає в рантаймі на Prisma або bcrypt.

Причина: немає `.dockerignore` → `COPY . .` затирає залежності, встановлені в контейнері, локальними.

Урок: `.dockerignore` пишеться **разом** із Dockerfile, а не потім.

4. Порт бази прокинутий на хост

Симптом: жодного — усе працює. Це й робить помилку небезпечною.

Причина: `ports: "5432:5432"` скопійовано з якогось туторіалу, де це було потрібно для підключення клієнтом.

Урок: кожен рядок `ports` — це свідоме рішення «сюди можна ходити ззовні». Треба підключитись клієнтом на хвилинку — прокидай порт тимчасово, а не назавжди у комітнутому файлі.

14 Шпаргалка

Щоденні команди

```
# Підняти все
docker compose up -d --build

# Подивитись логи одного сервісу
docker compose logs -f backend

# Зупинити (дані бази лишаються)
docker compose down

# Зупинити і ВИДАЛИТИ дані – так перевіряють чистий старт
docker compose down -v

# Що взагалі запущено і які порти стирчать назовні
docker compose ps

# Зайти в базу psql-ом (портів назовні немає – тільки так)
docker compose exec postgres psql -U postgres -d blog
```

Бекенд і Prisma

```
npm run dev           # локально з автоперезапуском
npm test              # юніт-тести
npm run test:integration # інтеграційні (сам підніме базу)
npm run test:all       # перед комітом

npx prisma migrate dev --name add_something # змінив схему
npx prisma studio           # візуальний перегляд даних у браузері
```

Перевірка, що все справді працює

```
# редірект з http на https
curl -sI http://localhost/ | head -1      # → 301

# https відповідає (-k бо сертифікат самопідписаний)
curl -sk https://localhost/api/health    # → {"status":"ok"}

# порт бази НЕ прокинутий – має бути порожньо
docker compose port postgres 5432
```

Куди рухатись далі

Тема	Навіщо саме тут
Валідація вхідних даних (zod)	зараз можна створити пост із порожнім заголовком — база це дозволить
Refresh-токени	щоб не тримати доступ 24 години і мати можливість відкликати сесію
Пагінація <code>/api/posts</code>	на 10 000 постів поточний запит поверне всі одразу
Rate limiting	<code>/auth/login</code> зараз можна перебирати без обмежень
Секрети поза compose	<code>JWT_SECRET</code> лежить у файлі, який іде в git
CI (GitHub Actions)	тести вже є — лишилось запускати їх автоматично на кожен PR

Головна думка

Жоден з інструментів тут не взято «бо модно». Express — щоб не парсити HTTP руками. Prisma — щоб структура бази версіювалась разом із кодом. Docker — щоб «у мене працює» перестало бути аргументом. NGINX — щоб Node не займався TLS і статикою. Тести — щоб правила доступу не зламались тихо. Коли додаєш наступний інструмент, спитай себе те саме: **яку конкретну проблему він знімає** — і чи є вона в тебе.